

Sequence analysis

PuffAligner: a fast, efficient and accurate aligner based on the Pufferfish index

Fatemeh Almodaresi [†], Mohsen Zakeri [†] and Rob Patro^{*}

Computer Science Department, University of Maryland, College Park, MD 20742, USA

^{*}To whom correspondence should be addressed.

[†]The authors wish it to be known that, in their opinion, the first two authors should be regarded as Joint First Authors.

Associate Editor: Janet Kelso

Received on October 8, 2020; revised on April 30, 2021; editorial decision on May 14, 2021

Abstract

Motivation: Sequence alignment is one of the first steps in many modern genomic analyses, such as variant detection, transcript abundance estimation and metagenomic profiling. Unfortunately, it is often a computationally expensive procedure. As the quantity of data and wealth of different assays and applications continue to grow, the need for accurate and fast alignment tools that scale to large collections of reference sequences persists.

Results: In this article, we introduce PuffAligner, a fast, accurate and versatile aligner built on top of the Pufferfish index. PuffAligner is able to produce highly sensitive alignments, similar to those of Bowtie2, but much more quickly. While exhibiting similar speed to the ultrafast STAR aligner, PuffAligner requires considerably less memory to construct its index and align reads. PuffAligner strikes a desirable balance with respect to the time, space and accuracy tradeoffs made by different alignment tools and provides a promising foundation on which to test new alignment ideas over large collections of sequences.

Availability and implementation: All the data used for preparing the results of this paper can be found with 10.5281/zenodo.4902332. PuffAligner is a free and open-source software. It is implemented in C++14 and can be obtained from <https://github.com/COMBINE-lab/pufferfish/tree/cigar-strings>.

Contact: rob@cs.umd.edu

Supplementary information: [Supplementary data](#) are available at *Bioinformatics* online.

1 Introduction

Since its introduction, next generation sequencing (NGS) has been widely used as a low-cost and accessible technology to produce high-throughput sequencing reads for many important biological assays.

The sequencing data that is generated in the form of short reads, drawn from longer molecular fragments, and finding the optimal alignments of these short reads to some reference is a necessary first step for many downstream biological analyses. The process of finding the segment on the reference that is most similar to the query read, and therefore most likely to be the source of the fragment from which the read was drawn, is known as read mapping or read alignment.

The main goal in read alignment is to find alignments of contiguous sub-string of the underlying reference that yields a minimum edit distance (or maximum alignment score) between the read and the reference sequence at the alignment position. If the reads are paired-end, characteristics other than the alignment score can be used to filter spurious alignment locations, such as orientation of each end of the alignment pair (forward or reverse) or distance between the

alignments corresponding to reads that are ends of the same fragment.

Short-read aligners are a major workhorse of modern genomics. Given the importance of the alignment problem, a tremendous number of different tools have been developed to tackle this problem. Some widely used examples are BWA (Li and Durbin, 2009), Bowtie2 (Langmead and Salzberg, 2012), Hisat2 (Kim *et al.*, 2015, 2019) and STAR (Dobin *et al.*, 2013).

Existing alignment tools use a variety of indexing methods. Some tools, such as BWA, Bowtie2 and STAR use a full-text index over the reference sequences; BWA and Bowtie2 use variants of the FM-index, while STAR uses a suffix array.

A popular alternative approach to full-text indices is to instead, index sub-strings of length k (k -mers) from the reference sequence. Trading off index size for potential sensitivity, such indices can either index all of the k -mers present in the underlying reference, or some uniform or intelligently chosen sampling of k -mers. There are a large variety of k -mer-based aligners, including tools like the Subread aligner (Liao *et al.*, 2013), SHRiMP2 (David *et al.*, 2011), mrfast (Alkan *et al.*, 2009) and mrsfast (Hach *et al.*, 2010). To reduce the index size, one can choose to select specific k -mers based

on a winnowing (or minimizer) scheme. This approach has been particularly common in tools designed for long-read sequence alignment like mashmap (Jain *et al.*, 2018) and minimap2 (Li, 2018).

Recently, a set of new indices for storing k -mers have been proposed based on graphs, specifically de Bruijn graphs (dBG). A de Bruijn graph is a graph over a set of distinct k -mers where each edge connects two neighboring k -mers that appear consecutively in a reference sequence and therefore, overlap on ' $k-1$ ' bases. Kallisto (Bray *et al.*, 2016), deBGA (Liu *et al.*, 2016), BGreat (Limasset *et al.*, 2016), BrownieAligner (Heydari *et al.*, 2018) and Pufferfish (Almodaresi *et al.*, 2018) are some tools which use an index constructed over the de Bruijn graph built from the reference sequences. Cortex (Iqbal *et al.*, 2012), Vari (Muggli *et al.*, 2017), rainbowfish (Almodaresi *et al.*, 2017) and mantis (Pandey *et al.*, 2018) are also tools that use a colored compacted de Bruijn graph for building their index over a set of raw experiments. All these approaches cover a wide range of the possible design space, and different design decisions yield different performance tradeoffs.

Generally, the fastest aligners (like STAR) have very large memory requirements for indexing, and make some sacrifices in sensitivity to obtain their speed. On the other hand, the most sensitive aligners (like Bowtie2) have very moderate memory requirements but obtain their sensitivity at the cost of a higher runtime. Maintaining the balance between time and memory is especially more critical while aligning to a large set of references, like a large collection of microbial and viral genomes which may be used as an index in microbiome or metagenomic studies. As both the collection of reference genomes and the amount of sequencing data grows quickly, it is important for alignment tools to scale to such large collections of data and references with reasonable resource requirements while remaining fast and sensitive.

Based on the compact Pufferfish (Almodaresi *et al.*, 2018) index, we introduce a new aligner called PuffAligner, that we believe strikes an interesting and useful balance in this design space. PuffAligner is designed to be a highly sensitive alignment tool while, simultaneously, placing a premium on computational overhead. By using the colored compacted de Bruijn graph to factor out repeated sub-sequences in the reference, it is able to leverage the speed and cache friendliness of hash-table-based aligners while still controlling the growth in the size of the index; especially in the context of redundant reference sequences. Therefore, the index provides favorable scalability, in terms of index size, construction memory and time, compared to popular indexes such as Bowtie2 and STAR. By carefully exploring the alignment challenges that arise in different assays, including single-organism DNA-seq, RNA-seq alignment to the transcriptome, and metagenomic sequencing, we have engineered a versatile tool that strikes desirable balance between accuracy, memory requirements and speed. We compare PuffAligner to some other popular aligners and show how it navigates these different tradeoffs.

2 Materials and methods

PuffAligner is an aligner built on top of the Pufferfish indexing data structure. Pufferfish is a space-efficient and fast index for the colored compacted de Bruijn graph (ccdBg). A colored compacted de Bruijn graph is a graph whose vertices (strings) are the compacted non-branching paths of the underlying de Bruijn graph, with the restriction that each node also have the same color set (set of reference sequences in which it appears). The nodes in the colored compacted de Bruijn graph are referred to as unitigs. Each unitig can exist in multiple references and be mapped to a list of <reference ID, position orientation > tuples. The tuple describes the position and orientation with which the unitig subsequence appears in each reference. The basic query operation in the Pufferfish index is to query a k -mer from the input sequence against the index. Given this query, the Pufferfish index returns the unique position (and orientation) where this k -mer appears in the colored compacted de Bruijn graph (or a sentinel value if this k -mer does not occur). This match between the query and the graph can then be easily 'unpacked' into the implied list of matches with the underlying references by finding all

of the places that the matched unitig appears in the reference sequences.

The main advantage of an aligner based on a compacted sequence graph, such as PuffAligner, offers compared to aligners based on linear full-text indices such as the FM-index, is the reduction of overhead related to computations for repeated subsequences within or across multiple references. This benefit results by performing the basic matching and mapping preprocessing steps on the graph itself, and annotating the results with respect to each reference, rather than performing these operations on all reference sequences individually. While k -mer query is the basic operation performed by Pufferfish index, we actually do not query for all the k -mer matches directly in PuffAligner. Instead, starting from k -mer k_0 , we extend the initial k -mer match into a unique maximal exact match [uni-MEMs, as introduced by (Liu *et al.* (2016))] which is the longest match of the read and the unitig containing k_0 . This uni-MEM is later used to construct a Maximal Exact Matches (MEM), which is a Maximal Exact Match shared between the read and the reference. We utilize the uniqueness property of unitigs to reduce the operational overhead of MEM extension per each reference to only once for a unitig. We expand a uni-MEM to different MEMs by assigning the reference information of the uni-MEM's underlying unitig to the uni-MEM. Using this approach, a uni-MEM can be a part of multiple MEMs, just as a unitig can be a part of multiple references. During MEM assembly, we merge overlapping consequent uni-MEMs that are only separated because of small branched unitigs. In this way, we guarantee the accuracy, performance of later mapping steps, and obtain similar behavior that is expected from dealing with chains of MEMs in more traditional alignment approaches. Finally, rather than fully aligning each query sequence to the anchored position on the reference, only the sub-sequences from the query that are not part of the MEMs (exact matches) appearing within the current high-scoring MEM chain are aligned to the reference; we call this procedure between-MEM alignment. Each of these steps are explained in detail in the following sections.

2.1 Exact matching in the Pufferfish index

The Pufferfish index provides PuffAligner with an efficient index for k -mer lookup within a list of references such as a collection of transcripts, genes or genomes. Specifically, the core components of the index are (i) a minimal perfect hash function (MPHF), (ii) a unitig sequence vector, (iii) a unitig-to-reference table and (iv) a vector storing the position associated with each k -mer in the unitig sequence vector. The unitig sequence vector contains all the unitigs in the ccdBg. In the Pufferfish index, a unitig is a monochromatic path in the colored compacted de Bruijn graph, where a color corresponds to a set of reference identifiers. Hence, each unitig is a substring of some set of reference sequences. Each unitig is annotated with the list of references that contain the unitig's sequence. In this design, a unitig can belong to multiple references and a reference can contain multiple unitigs.

The Pufferfish index admits efficient exact search for k -mers, as well as longer matches that are unique in both the query string and colored compacted de Bruijn graph. These matches, called uni-MEM, were originally defined in deBGA (Liu *et al.*, 2016). A uni-MEM is a Maximal Exact Match (MEM) between the query sequence and a unitig. Using the combination of the MPHF and the position vector, a k -mer is mapped to a unitig in the unitig sequence vector. The k -mer is then extended to a uni-MEM via a linear scan of the query sequence and the unitig sequence vector. Each uni-MEM can appear in multiple different references, and since uni-MEMs must be completely contained within a unitig, it is possible for multiple uni-MEMs to be directly adjacent on both the query and some references where the unitig appears.

uni-MEM collection: The first step in read alignment is to collect exact matches shared between the query (single-end or paired-end reads) and the reference. In PuffAligner, this is accomplished by collecting the set of uni-MEMs that co-occur between the query and reference. PuffAligner starts processing the read from the left-end and looks up each k -mer that is encountered until a match to the index is found. Once a match is discovered, it is extended in both query and the reference until one of these termination conditions occur: (i) a mismatch is encountered, (ii) the end of the query is

reached or (iii) the end of the unitig is reached. This process results in a uni-MEM match shared between the query and reference. uni-MEMs where extension is terminated as a result of reaching the end of a unitig must later be examined and potentially ‘collapsed’ together to form MEMs with respect to the references on which they appear. If the uni-MEM extension is not terminated as a result of reaching the end of the query, then the position in the read is incremented by a small value and the same procedure is repeated for the next k -mer on the read. This process continues until either the uni-MEM extension terminates because the end of the query is reached, or because the last k -mer of the query is searched in the index. Here, we recall an important property of uni-MEM extension that is different from e.g. MEM extension or maximum mappable prefix (MMP) extension (Dobin et al., 2013). Due to the definition of the ccdBg, it is guaranteed that any k -mer appearing within a uni-MEM cannot appear in any other unitig in the ccdBg. Thus, extending k -mers to maximal uni-MEMs is safe with respect to greedy extension, as such extension will never cause missing a k -mer that would lead to another distinct uni-MEM shared between the query and reference. The concept of safe extension of kmer matches was introduced in (Sarkar et al., 2018).

Filtering highly repetitive uni-MEMs: In order to avoid expending computation on performing the subsequent steps on regions of reads mapping to highly repeated regions of the reference, any uni-MEM that appears more than a user-defined number of times in the reference is discarded. In this manuscript, we use the threshold of 1000. This filter has a strong impact on the performance, since, even if one k -mer from the read maps to a highly repetitive region of the reference, the following expensive steps of the alignment procedure should be performed for every mapping position of the uni-MEM to find the right alignment for the read, while the less repetitive uni-MEMs also map to the true origin of the read on the reference as well. The drawback of this filter is that for a very small fraction of the reads which are truly originating from a highly repetitive region, all of the matched uni-MEMs will be filtered out and no hit remains for aligning the read. However, we find that in the case of aligning paired-end reads, usually one end of the read maps to a non-repetitive region, then, the alignment of the other end can be recovered using orphan recovery (explained in Section 2.4). Furthermore, we also provide a flag `-allowHighMultiMappers` that mitigates the effect of this filter for a slight tradeoff on the alignment performance.

uni-MEM compaction: For paired-end reads, PuffAligner aligns each end of the read pairs individually. For each end, all the uni-MEMs are sorted on the basis of their positions on the reference. First the uni-MEMs are ‘unpacked’ to their corresponding unitig’s reference list and assigned a reference position by adding the unitig’s relative position in the reference to the uni-MEM’s position in the unitig. After mapping the uni-MEMs to the references, it is now possible to construct the $\langle \text{read end}, \text{reference} \rangle$ MEMs. Each MEM is defined as either a single uni-MEM or a combination of those consecutive uni-MEMs that have an overlap of $k-1$ on both the query and reference (which is equal to unitigs’ overlap).

The compactable uni-MEMs result from terminating the extension process due to reaching the end of a unitig. After the compaction of uni-MEMs, there is a list of MEMs which are shared sequences between the query and a set of reference positions, that are sorted based on the reference positions.

Generally, a MEM is calculated for each pair of read and reference. This means that, in the case of highly similar references, the process of finding the same MEM of the read has to be repeated for each reference. Through the indirect construction of MEMs from uni-MEMs we gain a performance boost by extending the k -mer once per read and simply using the extended subsequence in MEMs of the read to difference references. However, the shorter the unitigs, the shorter the uni-MEMs. Therefore, the more k -mers per read need to be queried. This will affect the performance of MEM construction which, in the worst-case scenario, is required to query for each k -mer in the read. However, the k -mer queries still only happen once per each read and not once per each pair of read and reference.

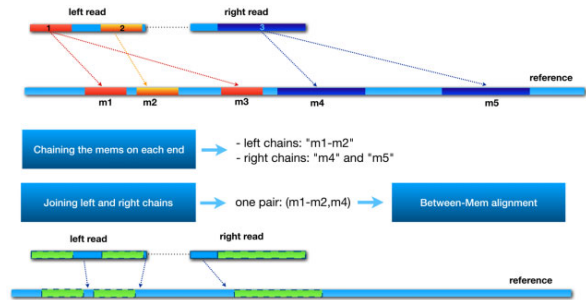


Fig. 1. This figure shows the main steps of chaining and between-MEM alignment in the PuffAligner procedure via an example. In this example, m1, m2 and m3 are the projected MEMs from the left end of the read to the reference and m4 and m5 are the projected MEMs from the right end of the read. In the first step, the chaining algorithm chooses the best chain of MEMs that provide the highest coverage score for each end of the read, that is the m1-m2 chain for the left end and two single MEM chain for the right end. Then, the selected chains from each end are joined together to find the concordant pairs of chains, that is the (m1-m2, m4) pair for this read as m5 is too far from m1-m2. Then, the chain from each end will go through to the next step, between-MEM alignment. For the green (dashed border) areas (MEMs) no alignment is recalculated as they are exact matches. Only the un-matched blue (solid border) parts of the reads (those nucleotides not occurring within a MEM) are aligned using a modified version of KSW2

2.2 Finding promising MEM chains

Via the procedures described above, we have enumerated all of the possible MEMs of length at least k shared between a read and the reference sequences. As shown in Figure 1, having all the MEMs from a read to each target reference, the goal of this step is to find promising chains of MEMs that cover the most unique bases in the read in a concordant fashion and that can potentially lead to a high quality alignment.

To accomplish this, we adopt the dynamic programming approach used in minimap2 (Li, 2018) for finding co-linear chains of MEMs that are likely candidates to support high-scoring read alignments. As mentioned in minimap2, all the MEMs from a read r to the reference t , are sorted by the ending position of the MEMs on the reference. Then, this algorithm computes a score for each set of MEMs based on the number of unique covered bases in the read, the coverage score is also penalized by the length of the gaps, both in the read and reference sequence, between each consecutive pair of MEMs.

In PuffAligner, if the distance between two MEMs, m_1 and m_2 , on the read and the reference is d_r and d_t respectively, these two MEMs should not be chained together if $|d_r - d_t| > C$, where C is the maximum allowed gap. So, the penalization term, the β value in (Li, 2018), in the coverage score computation is modified accordingly to prevent pairing of such MEMs.

Also, unlike what is done in minimap2 (Li, 2018), rather than considering together the MEMs that are discovered on both ends of a paired-end read, we consider the chaining for each end of the read separately. This is done in order to make it easier to enforce the orientation consistency of the individual chains. Specifically, the chaining algorithm that is presented in (Li, 2018) introduces a transition in the recursion that can be used to switch between the MEMs that are part of one read and those that are part of the other. However, such switching makes it difficult to enforce the orientation consistency of the chains that are being built for each end of the read. One solution to this problem is to add another dimension to the dynamic programming table, encoding if one has already switched from the MEMs of one read end to the other, and the recurrence can be modified to allow only one switch from the one read end to the other, allowing enforcement of orientation consistency. However, we found that, in practice, simply chaining the read ends separately led to better performance.

Finally, we also adopt the heuristic proposed by minimap2 (Li, 2018) when calculating the highest scoring chains. That is, when a MEM is added to the end of an existing chain, it is unlikely that a higher score for a chain containing this MEM will be obtained by

adding it to a preceding chain. Thus, we consider only a small fixed number of rounds (by default 2) of preceding chains once we have found the first chain to which we can add the current MEM.

The chaining algorithm described above finds the best chains of MEMs shared between the read r and the reference t in orientation o . A chain is accepted if its score is greater than a configurable fraction, which we call the *consensusFraction*, times the length of the read r . Throughout all the experiments in this manuscript the *consensusFraction* is set to 0.65. If a chain passes the consensus fraction threshold, we call it a *valid* chain. In addition, rather than keeping all valid chains, we also filter highly suboptimal chains with respect to the highest scoring chain *per-reference*. All valid chains shared between r and t are sorted by their scores, and chains having scores within 10% of the highest scoring chain for reference t are selected as potential mappings of the read r to the reference t . While these filters are essential for improving the throughput of the algorithm in finding the right alignment, they are carefully selected to have very little effect on the sensitivity of PuffAligner. For all the experiments in this manuscript, the same default settings of these parameters are used if not mentioned otherwise.

2.3 Computing base-to-base alignments between MEMs

After finding the high-scoring MEM chains for each reference sequence, a base-to-base alignment of the read to each of the candidate reference sequences is computed. Each selected chain implies a position on the reference sequence where the read might exhibit a high quality alignment. Thus, we can attempt to compute an optimal alignment of the read to the reference at this implied position, potentially allowing a small bit of padding on each side of the read. This approach utilizes the positional information provided by the MEM chains. However, the starting position of the alignments is not the only piece of information embedded in the chains. Rather each chain of MEMs consists of sub-sequences of the read (of size at least k , though often longer) which match exactly to the reference. While the optimal alignment of the read to the reference at the position being considered is not *guaranteed* to contain these exact matches as alignments of the corresponding substrings, this is almost always the case in practice.

In PuffAligner, we aim to exploit the information from the long matches to accelerate the computation of the alignments. In fact, since only chains with relatively high coverage score are selected, a large portion of the read sequences are typically already matched to the positions in the reference with which they will be matched in the final optimal alignment. For instance, in Figure 1, for the final chains selected on the reference sequence, it is already known for the red and orange sub-sequences (areas 1 and 2) on the left end of the read precisely where they should align to the reference. Likewise this is the case for the purple sub-sequence (area 3) on the right read. The unmapped regions of the reads are either bordered by the exact matches on both sides, or they occur at the either ends of the read sequence. In fact, other aligners such as STAR (Dobin *et al.*, 2013), BWA-MEM (Li and Durbin, 2009) and minimap2 (Li, 2018) employ the same strategy to assume that the exact matches, found during the query time, are part of the final alignments to be reported. PuffAligner skips aligning the whole read sequence by considering the exact matches of the MEMs to be part of the alignment solution. As a result, it is only required to compute the alignment of the small unmapped regions, which reduces the computation burden of the alignments.

When applying such an approach, two different types of alignment problems are introduced, which we call bounded sub-sequence alignment and ending sub-sequence alignment. For bounded sub-sequence alignment, we need to *globally* align some interval i_r of the read to an interval i_t of the reference. If i_r and i_t are of different lengths, the alignment solution will necessarily include insertions or deletions. If i_r and i_t are of the same length, then the optimal global alignment between them may or may not include indels. For each such bounded sub-sequence alignment, we determine the optimal alignment of i_r to i_t by computing a global pair-wise alignment between the intervals, and stitching the resulting alignment together with the exact matches that bound these regions.

Gaps at the beginning or the end of the read are symmetric cases and so we describe, without loss of generality, the case where there is an unaligned interval of the read after the last MEM shared between the read and the reference. In this case, we need to solve the ending sub-sequence alignment problem. Here, the unaligned interval of the read consists of the substring spanning from the last nucleotide of the terminal MEM in the chain, up through the last nucleotide of the read. There is not a clearly defined interval on the reference sequence. While the left end of the relevant reference interval is defined by the last reference nucleotide that is part of the bounding MEM, the right end of the reference interval should be determined by actually solving an extension or ‘end-free’ alignment problem. We address this by performing extension alignment of the unaligned interval of the read to an interval of the reference that begins on the reference at the end of the terminal MEM, and extends for the length of the unaligned query interval plus the length of some problem-dependent buffer (which is determined by the maximum length difference between the read and reference intervals that would still admit an alignment within the acceptable score threshold).

PuffAligner uses KSW2 (Li, 2018; Suzuki and Kasahara, 2018) for computing the alignments of the gaps between the MEMs and for aligning the ending sequences. KSW2 exposes a number of alignment modes such as global and extension alignments. For aligning the bounded regions, KSW2 alignment in the global mode is performed, and for the gaps at the beginning or end of reads, PuffAligner uses the extension mode to find the best possible alignment of that region. For increasing the efficiency of alignment computation we also employ some specific techniques in PuffAligner which are explained in Supplementary Section S1.

2.4 Joining mappings for read ends

Finally, once alignments have been computed for the individual ends of a read, they must be paired together to produce valid alignments for the entire fragment. At this point in the process, on each reference sequence, there are a number of locations where the left end of each read or the right end of each read, or both, are mapped to the reference. For the purpose of determining which mappings will be reported as a valid pair, the mappings are joined together only if they occur on opposite strands of the reference, and if they are within a maximum allowed fragment length. There are two different types of paired-end alignments that can be reported by PuffAligner; concordant and discordant. If PuffAligner is disallowed from reporting discordant alignments, then the mapping orientation of the left and right end should agree with the library preparation protocols of the reads. PuffAligner first tries to find concordant mapping pairs on a reference sequence, and if no concordant mapping is discovered and the tool is being run in a mode where discordant mappings are allowed, then PuffAligner reports pairs that map discordantly. Here, discordant pairs may be pairs that do not, for example, obey the requirement of originating from opposite strands. While this is not expected to happen frequently, it may occur if there has been an inversion in the sequenced genome with respect to the reference.

3 Results

For measuring the performance of PuffAligner and comparing it to other aligners, we have designed a series of experiments using both simulated and experimental data from different sequencing assays. We compare PuffAligner with Bowtie2 (Langmead and Salzberg, 2012), STAR (Dobin *et al.*, 2013) and deBGA (Liu *et al.*, 2016). Bowtie2 is a popular, sensitive and accurate aligner with the benefit of having very modest memory requirements. STAR requires a much larger amount of memory, but is much faster than Bowtie2 and can also perform ‘spliced alignment’ against a reference (which PuffAligner, Bowtie2 and deBGA currently do not allow). deBGA, is most-related tool to PuffAligner conceptually, as it is an aligner with a colored compacted de Bruijn graph-based index that is focused on exploiting redundancy in the reference sequence.